

第9章 归约

9.1 引言

- 定义：将问题 P 转化为问题 Q ，分析或求解问题 Q ，并根据问题 Q 的解回答问题 P
- 用途：根据问题 Q 求解问题 P ；通过分析问题 P 的复杂度下界，推知问题 Q 的复杂度下界

9.2 循环平移问题

- 问题：判断等长的字符串之间是否为循环平移，如 $abcde$ 和 $cdeab$ 互为循环平移
- 直接法：截断一个字符串为两个字串，然后多次KMP/BM
- 归约法：将一个字符串复制一份，拼接到原字符串后面，然后通过一次KMP/BM判断另一个字符串是否为拼接字符串的子串

9.3 代表集问题

- 问题：给定 k 个集合 $S_1, S_2, S_3, \dots, S_k$ 及各集合的元素，定义代表集 R 为 $R = \{r_1, r_2, r_3, \dots, r_k | r_i \in S_i\}$ ，求给定的若干集合是否存在代表集
注意：因为 $r_i \in R, r_j \in R$ ，所以 $r_i \neq r_j$
- 定理： $S_1, S_2, S_3, \dots, S_k$ 存在代表集 $\Leftrightarrow$

$$\forall I \subset \{1, 2, 3, \dots, k\}, |I| = m, \left| \bigcup_{j=1}^m S_{i_j} \right| \geq m$$

- 作为算法的缺点： $\{1, 2, 3, \dots, k\}$ 的子集数为 2^k ，算法为指数时间
- 归约：（偶图匹配）设 $G = (V, W, E)$ 为二分图，将集合映射到 V 中顶点，将元素映射到 W 中顶点，求二分图的匹配的过程，即为从每个集合中选出元素组成代表集的过程

9.4 序列比较问题

- 问题与动态规划解法参见第5章5.9节
- 归约：将动态规划表视为有向图，求最短路径问题，则算法的时间复杂度会增大，但可解决问题的类型会增多
比如在不同位置增删改的代价不同时，动态规划较为困难，可归约为最短路径问题解决

9.5 无向图的完全子图

- **问题：**寻找 $G = (V, E)$ 中是否包含 3 个顶点的完全子图（即三角形）

- **暴力遍历：** $\Theta\left(\binom{n}{3}\right) = \Theta(n^3)$

- **归约：**设 A 为 G 的邻接矩阵，则 A 为对称布尔阵

根据矩阵乘法定义，有 $A^2[i, j] = \sum_{k=1}^n A[i, k]A[k, j]$

因此若 $A^2[i, j] = 1$ ，则 $\exists k, A[i, k] = A[k, j] = 1$ ，此时只需 $A[i, j] = 1$ 则存在三角形 i, j, k

布尔矩阵乘法时间复杂度：优化前 $O\left(\frac{n^3}{\log n}\right)$ ，优化后 $O\left(\frac{n^3}{\log^2 n}\right)$ ，均快于暴力遍历

9.6 线性规划

- **模型：**将目标问题归约为如下种类的问题：在满足等式约束，不等式约束，变量约束条件下，如何使目标函数取到最值

9.6.1 网络流量归约

- **问题：**有向图 $G = (V, E)$ 含有入度为 0 的结点 s ，出度为 0 的结点 t ，每条边有权值 $w(e)$ 表示 e 的最大流量，满足每条边的流量 $f(e) \leq w(e)$ ，除 s, t 外每个结点的流入量等于流出量，求网络中流量的最大值（即流入 s 或流出 t 的流量的最大值）

- **线性规划：**

- 目标函数： $c(x) = \sum_{i \in S} x_i$ ，其中 S 为 s 关联的边集
- 约束不等式： $\forall i, x_i \leq w(x_i)$
- 约束等式： $\sum_{i \in V_{in}} x_i = \sum_{i \in V_{out}} x_i$
- 变量约束： $x_i \geq 0$

9.6.2 静态路由归约

- **问题：**无向图 $G = (V, E)$ 中，每个结点传输信息速率有限（可收可发，结点有缓存），求每条边传输的消息速率，使网络中传输消息数量最多

- **线性规划：**设每条边传输速率 x_i

- 目标函数： $\sum_i x_i$
- 约束不等式： $\sum_{v_j} x_i \leq \max v_j$
- 变量约束： $x_i \geq 0$

9.7 下界的归约

9.7.1 问题的下界

- **问题的下界**：问题的下界即问题的复杂度，求解问题的算法复杂度不可能低于问题的复杂度，低于问题复杂度的算法一定是错误的
- **平凡下界**：不借助证明，仅借助合理性给出问题的复杂度下界
 - 求全序集的最小值，至少要遍历全集，所以问题下界 $\Omega(n)$
 - 求方阵乘法，至少结果方阵的规模为 n^2 ，所以问题下界 $\Omega(n^2)$
- **下界与归约**：若问题 P 的下界 $\Theta(f(n))$ ，且可根据问题 Q 的解改造为问题 P 的解，且改造算法的复杂度 $o(f(n))$ ，则问题 Q 的下界至少为 $\Theta(f(n))$

9.7.2 决策树模型

- **模型**：算法可以看作在一棵树中寻找问题解，所有可能的情况为树的叶结点
- **复杂性分析**：设 L 是叶结点集， $depth(v)$ 表示从根节点到结点 v 的路径长，即结点的深度，定义树的外路径总长 $EPL(T) = \sum_{x \in L} depth(x)$ ，则对于二叉决策树有 $EPL(T)_{min} > |L|(\lceil \log |L| \rceil - 1)$ ，当 L 确定时， $EPL(T)$ 最小情况是满二叉树（内结点有 2 个子结点，且叶结点集中在最下方 2 层）
- **应用**：（分析排序问题的下界）基于比较的排序，决策树叶结点个数 $|L| = n!$ ，平均至少需要 $EPL(T)_{min}/|L| = \log(n!) - 1$ 次比较，所以基于比较的排序算法复杂度均为 $\Omega(n \log n)$

9.7.3 简单多边形问题复杂度

- 简单多边形问题参见第7章7.3节
- 将排序问题中的点 θ_i 映射为 $(\cos \theta_i, \sin \theta_i)$
若可在 $o(n \log n)$ 时间内求出简单多边形，则也能在 $o(n \log n)$ 时间内排序，与排序问题复杂度 $\Omega(n \log n)$ 矛盾，因此简单多边形问题复杂度 $\Omega(n \log n)$

9.7.4 矩阵相乘下界归约

- 若可在 $O(T(n))$ 时间内计算 2 个 $n \times n$ 对称矩阵的乘积，根据定义易知 $T(n) = O(n^3)$ ，所以有 $T(2n) = O(T(n))$ ，则：可在 $O(T(n) + n^2)$ 内计算 2 个 $n \times n$ 任意矩阵的乘法证明：

$$\begin{pmatrix} O & A \\ A^T & O \end{pmatrix} \begin{pmatrix} O & B^T \\ B & O \end{pmatrix} = \begin{pmatrix} AB & O \\ O & A^T B^T \end{pmatrix}$$

扩展： $O(n^2)$

乘法： $O(T(2n)) = O(T(n))$

提取结果： $O(n^2)$

总复杂度： $O(T(n) + n^2)$

- 若可在 $O(T(n))$ 时间内计算 $n \times n$ 对称矩阵的平方，根据定义易知 $T(n) = O(n^3)$ ，所以有 $T(2n) = O(T(n))$ ，则：可在 $O(T(n) + n^2)$ 内计算 2 个 $n \times n$ 任意矩阵的乘法
证明：

$$\begin{pmatrix} O & A \\ B & O \end{pmatrix}^2 = \begin{pmatrix} AB & O \\ O & BA \end{pmatrix}$$

扩展： $O(n^2)$

乘法： $O(T(2n)) = O(T(n))$

提取结果： $O(n^2)$

总复杂度： $O(T(n) + n^2)$